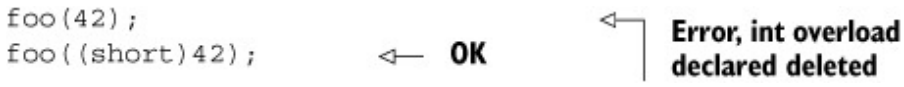
You can apply the `= delete` specifier to any function, not just copy constructors and assignment operators. This makes it clear that the function isn't available. It does a bit more than that too, though; a deleted function participates in overload resolution in the normal way and only causes a compilation error if it's selected. This can be used to remove specific overloads. For example, if your function takes a `short` parameter, you can prevent narrowing of `int` values by writing an overload that takes an `int` and declaring it deleted:

```
void foo(short);
void foo(int) = delete;
```

Any attempts to call `foo` with an `int` will now be met with a compilation error, and the caller will have to explicitly cast supplied values to `short`:



A.3. Defaulted functions

Whereas deleted functions allow you to explicitly declare that a function isn't implemented, *defaulted* functions are the opposite extreme: they allow you to specify that the compiler should write the function for you, with its "default" implementation. Of course, you can only do this for functions that the compiler can autogenerate anyway: default constructors, destructors, copy constructors, move constructors, copy-assignment operators, and move-assignment operators.

Why would you want to do that? There are several reasons why you might:

- *In order to change the accessibility of the function*—By default, the compiler-generated functions are `public`. If you wish to make them `protected` or even `private`, you must write them yourself. By declaring them as defaulted, you can get the compiler to write the function *and* change the access level.
- *As documentation*—If the compiler-generated version is sufficient, it might be worth explicitly declaring it as such so that when you or